Quick Sort Analysis



Algorithm Design Technique

Divide and Conquer

Divide and Conquer

- It is often easier to *solve several small instances* of a problem than one large one.
  - *divide* the problem into smaller instances of the same problem
  - solve (*conquer*) the smaller instances recursively
  - *combine* the solutions to obtain the solution for original input

- Solve(I)
  - n = size(I)
  - if (n <= smallsize)
    - solution = directlySolve(I);
  - else
    - divide I into I1, , Ik.
    - for each i in {1, , k}
      - Si = solve(Ii);
    - solution = combine(S1, , Sk);
  - return solution;

```
QUICKSORT(A, p, r)
if  p< r
    then q←PARTITION(A, p, r)
QUICKSORT(A, p, q-1)
QUICKSORT(A, q+1, r)
```

Versions Of Quick sort

- Randomized Quick sort
- Median of 3 Partition

PARTITION (A,p,r)

- x=A[r]
- i ← p -1
- For j ← p to r-1
- do if A[j] <= x
- then i ← i + 1
- exchange A[i] <-> A[j]
- Exchange A[i+1] <-> A[r]
- Return i+1

Ex: 5 7 6 1 3 2 4

Initial call:QUICKSORT(*A*, p, *r*)
➜ QUICKSORT(*A*, 1, *7*)

Versions Of Quick sort

Randomized Quick sort

RANDOMIZED-QUICKSORT(*A*, *p*, *r*)
if *p*< *r*
then *q* ← RANDOMIZED-PARTITION(*A*, *p*, *r*)
RANDOMIZED-QUICKSORT(*A*, *p*, *q*–1)
RANDOMIZED-QUICKSORT(*A*, *q*+1, *r*)

RANDOMIZED-PARTITION (A, p, r)
- i ← RANDOM (p, r)
- exchange A[r] <-> A[i]
- return PARTITION (A, p, r)

RANDOM(a,b) returns a random integer in the range (a,b) i.e. (b-a+1) integers

Median Method V2- Quick Sort

Partition

- 1. Choose a random pivot element
- 2. swap it with last index element
- 3. (for i) proceed from left (i++) till elements are less than pivot [also (i<j)]
- 4. (for j) proceed from right (j--) till elements are more than pivot [also (i<j)]
- 5.exchange i & j elements when condition is unsatisfied [keeping (i<j)]
- 6. for (i>j) swap i with last element

Median Method V2- Quick Sort

Partition

- 1. Choose a random pivot element
- 2. swap it with last index element
- 3. (for i) proceed from left (i++) till elements are less than pivot [also (i<j)]
- 4. (for j) proceed from right (j--) till elements are more than pivot [also (i<j)]
- 5.exchange i & j elements when condition is unsatisfied [keeping (i<j)]
- 6. for (i>j) swap i with last element

Ex.

5 7 6 1 3 2 4

MEDIAN-OF-3-PARTITION

Median-of-3-Partition (A, p, r)

```
i ← RANDOM (p, r)
j ← RANDOM (p, r)
k ← RANDOM (p, r)

if(A[i]<=A[j] and A[j]<=A[k]) or (A[k]<=A[j] and A[j]<=A[i])
     then l ← j
else if  if(A[j]<=A[i] and A[i]<=A[k]) or (A[k]<=A[i] and A[i]<=A[j])
     then l ← i
else if (A[i]<=A[k] and A[k]<=A[j]) or (A[j]<=A[k] and A[k]<=A[i])
     then l ← k

exchange A[r] <-> A[l]
return PARTITION (A, p, r)
```

This approach tends to choose the pivot value as median of 3 elements selected randomly. In general this algorithm gives very good results except for few cases

Median-of-3-Partition (A, p, r)

Generally, with any value of i, j, k it will dilute the worst sequence

Consider any worst case sequence

23, 67, 12, 89, 34, 95



Performance of Quick Sort

Depends on whether partitioning is balanced or unbalanced

- If it is balanced :
  - Runs asymptotically as fast as merge sort
- If it is unbalanced
  - Runs asymptotically as slow as insertion sort

Worst Case Partitioning

When it produces one sub problem with n-1 elements and one with zero elements

Lets us assume unbalanced partitioning, the partitioning cost $\theta(n)$ time

The recurrence for running time is

$$T(n) = T(n-1) + T(0) + \theta(n)$$
$$= T(n-1) + \theta(n)$$

Using recurrence Method we get $\theta(n^2)$

Note : It occurs when input array is already sorted , the best case condition for insertion sort

Best Case Partitioning

- The recurrence for running time is

- $T(n) <= 2T(n/2) + \theta(n)$

Using Main recurrence theorem we get

$T(n) = \theta(n \log n)$

- When the partitioning splits into some ratio:

$T(n) = \Theta(n \log n)$

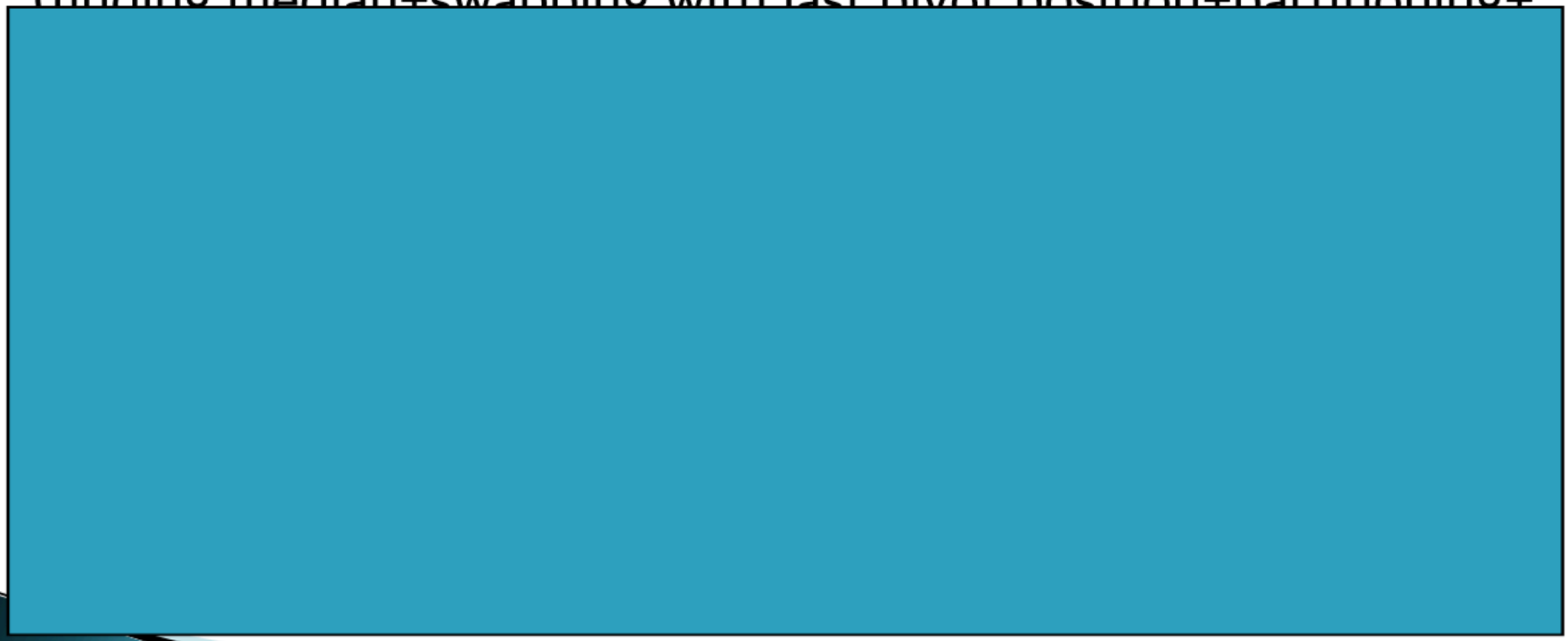
- When the partitioning split alternative best and worst segments

$T(n) = \Theta(n \log n)$

The median of ‘n’ elements can be found in O(n) time.

What is the complexity of quicksort, in which medium is selected as pivot ?

(finding median+swapping with last pivot position+partitioning+



The median of ‘n’ elements can be found in O(n) time

What is the complexity of quicksort, in which medium is selected as pivot ?

(finding median + swapping with last pivot position + partitioning + division into equal halves)

$T(n) = O(n) + O(1) + O(n) + 2\,T(n/2)$

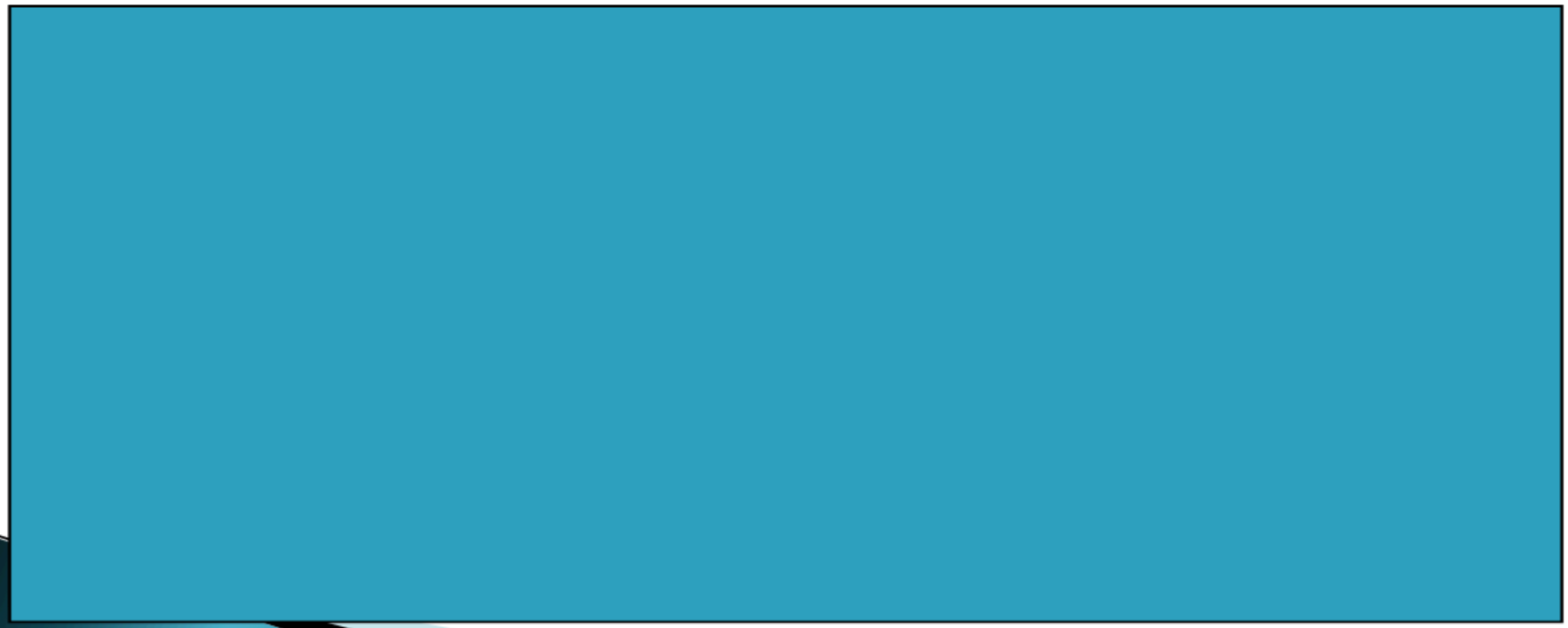
$= O(n) + 2\,T(n/2)$

$= O(n \log n)$

In quick sort, for sorting ‘n’ elements, the (n/4)th smallest element is selected as pivot using O(n) time algorithm.

What is the worst case complexity of quick sort?

(element selection, swapping with pivot position, two



In quick sort, for sorting 'n' elements, the (n/4)th smallest element is selected as pivot using O(n) time algorithm.

What is the worst case complexity of quick sort?

(element selection + swapping with pivot + partition+ two partition in ration 1:4)

$T(n)=O(n)+O(1)+O(n)+T(n/4)+T(3n/4)$

$= O(n) + \theta (n \log n)$

$=\theta (n \log n)$

Let T1 and T2 be the time taken partitioning the following sequences

1,2,3,4,5……..n T1

n,n-1,n-2…….1 T2

Find the relation between T1 and T2 ?



Let T1 and T2 be the time taken partitioning the following sequences

1,2,3,4,5……..n T1

n,n-1,n-2…….1 T2

Find the relation between T1 and T2 ?

Ans : **T1=T2**

- Space Complexity

(defined by number of levels in recursive tree)

- For Balanced partition = O(log n)

- For unbalanced partition = O(n)

Video Lecture of the topic can be found at

Divide And Conquer Strategy of problem Solving

https://youtu.be/HO4B-kG_6Ag

Quick Sort Analysis

https://youtu.be/LPE9xYEsfIU

Quick Sort Analysis for various partitioning scenarios

https://youtu.be/gCP06CTpEaE

Understanding Space Complexity

https://youtu.be/HEZdrU8ZPIk

https://cexpertvision.com/2020/10/18/comparison-based-sorting-ca/